GFlowQEC: Generative Flow Networks that Learn to Sample Quantum LDPC Codes*

Nuno Batista[1], Gabriel Falcão[2,3], and Penousal Machado[4]

[1] Department of Informatics Engineering, University of Coimbra, Coimbra, Portugal
[2] Instituto de Telecomunicações, Department of Electrical and Computer Engineering, University of Coimbra, Coimbra, Portugal
[3] New York University Abu Dhabi, Abu Dhabi, United Arab Emirates
[4] University of Coimbra, CISUC/LASI – Centre for Informatics and Systems of the University of Coimbra, Department of Informatics Engineering

nuno.batista@uc.pt, gff9537@nyu.edu, machado@dei.uc.pt

Abstract. Choosing a quantum error-correcting code for a real device means scoring candidates with circuit-level simulations so slow that only a handful can ever be run. So every evaluation counts, and a search is judged by how many good codes it finds per evaluation it pays for. Quantum LDPC codes built from a few elements of a finite group make this hard: they can protect many logical qubits at once, but good ones are rare in a huge space. We introduce GFlowQEC, to our knowledge the first generative flow network (GFlowNet) for quantum code design. It builds a code one group element at a time and learns to propose codes in proportion to a power of their score. We train it once on scores from earlier searches, freeze it, and let it search lifted-product codes with 180 physical qubits. Per paid call to the scoring oracle, it finds more than twice as many new top-scoring codes (inequivalent to any it was trained on) as tuned parallel tempering, even when tempering starts from GFlowQEC's training codes. It finds more than three and a half times as many as a supervised screen that ranks ten random candidates per call. A screen that ranks a hundred beats GFlowQEC, and does best when it ranks GFlowQEC's draws instead. Soft Q-learning, which shares the GFlowNet's optimal policy, does as well at one noise bias and better at the other, so the gain comes from learning to propose, not from the GFlowNet objective. Trained proposers can make a budget of expensive evaluations go much further when good codes are rare.

Keywords: Quantum error correction · Quantum LDPC codes · Generative flow networks · Code search

* This work is funded by national funds through FCT – Foundation for Science and Technology, I.P., within the scope of the research unit UID/00326 - Centre for Informatics and Systems of the University of Coimbra, https://doi.org/10.54499/UID/00326/2025. This work is funded by national funds through FCT – Fundação para a Ciência e a Tecnologia, I.P., and, when eligible, co-funded by EU funds under project/support UID/50008/2025 – Instituto de Telecomunicações, with DOI identifier https://doi.org/10.54499/UID/50008/2025.

1 Introduction

Quantum low-density parity-check (LDPC) codes can protect many logical qubits in a single block of physical qubits [30,6]. Many of them are specified by a short list of elements of a finite group, and since every list gives a valid code [28,37], designing one amounts to a search over lists. That search is hard for two reasons. The space is enormous: the $[[300, 60, 14]]$ mitten code of Bhardwaj et al. [4] is a list of ten elements of a group with sixty elements, and lists of that length give up to about 3×10^8 inequivalent codes. And every candidate has to be scored, either by computing its distance or by simulating how often it fails, which takes from about a second to a few minutes per code. A search therefore only ever sees a sliver of the space.

Our motivating setting is choosing a code for a particular device. There, each candidate is scored by a circuit-level simulation, with faulty measurements and a real decoder, which takes minutes to hours per code, so only a handful are ever scored [52,5,4]. Since the best code can also change with the device's noise and connectivity [5,3], a cheaper score can only shortlist candidates for the expensive simulation, so a search should deliver many good candidates for its budget rather than a single best one [1]. Because methods cannot yet be compared fairly at that price, we compare them where scoring is cheap enough to run each method many times, but we keep the unit of cost that matters on a device: the paid call, that is, one call to the scoring routine (the oracle) for a code the run has not scored before.

Bayesian optimisation, reinforcement learning, filtered uniform sampling and large language models (LLMs) have already been used to search code families of this kind, and Table 1 compares these searches with ours. None of them, however,

Table 1. Prior code searches compared with this paper. BB stands for bivariate bicycle and LP for lifted product.

Work	Family	Score	Returns	Matched budget	Exact distances
Chengyu et al. [10], Bayesian optimisation	BB, any weight	Rate k/n and error rate	one code per weighting	yes	no
Cruz-Benito et al. [12], LLM	BB	kd^2/n	a catalog	partly	some
Liu and Marquardt [29], LLM	LP	Rate k/n and error rate	an archive	no	no
Bhardwaj et al. [4], filtered sampling	LP, non-abelian	Distance upper bounds	one code per group	no	some
Sella et al. [43], reinforcement learning	BB, elements $(i, 0)$ or $(0, j)$	Decoder success rate	best code so far	no	no
Freire et al. [15], three searches	Hypergraph product	Erasure failure rate	one best code	yes	not stated
This paper, GFlowNet policy	Two-block and LP	Merit, failure exponent or error rate (Sec. 4.3)	samples, target proportional to reward	yes	some

trains a model to sample codes in proportion to their quality.

Learning can turn a fixed budget into more good codes in two ways. A screen draws random candidates and pays only for the ones a learned model ranks highest, whereas a proposer learns to draw good candidates directly. When good codes are rare, the difference matters: a screen has to look at many random candidates for every one it keeps, while a proposer builds that rejection into its own draws. Generative flow networks (GFlowNets) [1,2] are proposers of this kind. A GFlowNet builds an object step by step and is trained so that, at its optimum, it produces each object with probability proportional to a reward, which means one model yields many distinct good objects where reward maximisation would converge on one [1]. In code search the draws then cover all good codes, the best ones most often, rather than only the code a cheap score ranks first. To our knowledge, no GFlowNet has yet been used to design quantum error-correcting codes.

This paper makes three contributions.

- We introduce GFlowQEC, a GFlowNet whose policy fills in the list one group element per step and is trained with trajectory balance [50] to produce each list with probability proportional to a power of its score.
- We test this policy on lifted-product codes with 180 physical qubits, where it is trained once on scores stored from earlier searches, frozen, and compared with tuned parallel tempering, a supervised screen and soft Q-learning at matched budgets of paid calls.
- We construct a benchmark on which any search method can be graded. It covers two code families, which include the bivariate bicycle [5] and mitten codes, at four sizes from 36 to 300 physical qubits, and it gives an exact distance for every code that encodes at least one logical qubit at the two smallest sizes.

At 180 physical qubits the frozen policy, GFlowQEC-offline, draws ten candidates for each paid call and pays for the ones it ranks most probable. Counting only codes inequivalent to those it was trained on, it found more than twice as many top-scoring codes per paid call as tuned parallel tempering (tuned PT), 2.4 and 2.8 times as many at the two noise biases. Started from the same stored codes, tuned PT did not close the gap, so the lead is not just access to the stored data. It also found more than three and a half times as many as a supervised screen built on the same network and trained on the same scores. Because only about one random code in a hundred is top-scoring here, a screen shown ten random codes per paid call rarely sees one. Shown a hundred random codes per paid call, the screen found more than the policy, and the largest count of all came from a screen that ranked the policy's own draws. Soft Q-learning trained in the same way, which shares the GFlowNet's optimal policy on our construction [47], did as well at one bias and better at the other, so the gain comes from training a proposer rather than from the GFlowNet objective.

2 Related work

Code families. The searches below use the name bivariate bicycle (Sec. 3.2), each in its own sense. The mitten codes [4] are lifted-product codes [37,28,36] over non-abelian groups, since over abelian groups their construction has distance at most 6.

Code search. Table 1 sets the closest searches beside ours. They use Bayesian optimisation [10], LLMs [12,29], reinforcement learning [43] and uniform sampling with algebraic filters, the method that found the mitten codes [4]. Simple baselines are strong in this setting: Freire et al. [15], who compare methods at a matched budget under one oracle as we do, find on hypergraph-product codes that a random walk matches simulated annealing and reinforcement learning. Cruz-Benito et al. compared their LLM search with uniform sampling at a comparable budget [12]. In a later study of weight-five codes [11], 87% of the LLM's proposals were certified at $d \geq 5$, against 73 to 74% of uniform samples of the same size, but the uniform samples reached the LLM's best code class and covered more classes. Beyond Table 1, LLM agents search under circuit-level noise [52] and over balanced-product codes [38]. Reinforcement learning has glued small codes into codes of up to 20 physical qubits with one logical qubit [45], as well as a code on nine physical qubits together with its encoder, with the noise bias as an input [34]. We do not reimplement the LLM searches [12,29,52,38], because their oracles and budgets differ from ours and their models may have seen published codes of these families during pretraining.

GFlowNets. On a tree, where every object has a single construction path, GFlowNet training is equivalent to entropy-regularised reinforcement learning such as soft Q-learning [47]. Conditional GFlowNets take a parameter of the reward as an input, such as the reward's temperature in Logit-GFN [23] or the noise bias in GFlowQEC-offline. Training a GFlowNet only on stored rewards, with no reward model and no new oracle call, is an established setting [8]. Like the plain baseline of that work, GFlowQEC-offline trains directly on its stored codes, and it adds conditioning on the bias and a deployment that ranks its own draws. GFlowNets have also been applied to quantum circuits: FlowQ-Net [13] builds them gate by gate and lists quantum error-correcting code synthesis among its future applications, and QFlowNet [26] applies a GFlowNet to unitary synthesis.

3 Background

3.1 Quantum LDPC codes

An error on a qubit is a bit flip (X), a phase flip (Z) or both (Y). A Calderbank-Shor-Steane (CSS) code [7,44] is specified by two binary matrices H_X and H_Z, each with one column per physical qubit. Each row is a check, that is, a parity

measurement on the qubits where the row has a 1, and the pair of matrices defines a valid code exactly when

$$H_X H_Z^{\mathsf{T}} = 0 \pmod 2. \tag{1}$$

The number of logical qubits is

$$k = n - \operatorname{rank} H_X - \operatorname{rank} H_Z, \tag{2}$$

with the ranks taken over $\mathbb{F}_2$, the field with two elements (arithmetic mod 2). A pattern of phase flips $e_Z \in \mathbb{F}_2^n$ marks the qubits that are hit, and its weight $|e_Z|$ is the number of such qubits. The X checks detect the pattern when $H_X e_Z \neq 0$, and the Z checks detect bit flips e_X in the same way. Throughout, a subscript names the kind of flip, so e_Z, d_Z and p_Z all refer to phase flips.

Sums of rows of H_Z, read as phase flips, are the stabilizers: they leave the code's states unchanged and are not seen by the X checks (Eq. (1)). Every other undetected pattern changes the encoded information and is a logical operator, and the lightest logical operator of each kind defines the distance,

$$\begin{aligned} d_Z &= \min\{\, |e_Z| : H_X e_Z = 0,\ e_Z \notin \operatorname{rowspace} H_Z \,\}, \\ d_X &= \min\{\, |e_X| : H_Z e_X = 0,\ e_X \notin \operatorname{rowspace} H_X \,\}, \\ d &= \min(d_X, d_Z). \end{aligned} \tag{3}$$

where $\operatorname{rowspace} H$ is the set of sums of rows of H. A code with these parameters is written $[[n, k, d]]$. Its Tanner graph has one vertex for each qubit and each check, and an edge wherever a check's row has a 1 (Fig. 1). For a quantum LDPC code this graph is sparse [30].

3.2 Codes from a group algebra: the families we use

In both families we use, a code is specified by a tuple x of six or ten elements of a group G, and every search method in this paper proposes such tuples. The groups are $Z_l \times Z_m$, whose elements are the pairs (i, j) of integers mod l and mod m, and $Z_l \times S_3$, where S_3 is the group of the six orderings of three objects. Of the two, $Z_l \times Z_m$ is abelian, so the order of a product does not matter, whereas $Z_l \times S_3$ is not. A sum such as $a = g_1 + g_2 + g_3$ is a set of group elements. Two sums multiply like polynomials, with every element of one multiplied by every element of the other, and a product that appears an even number of times cancels. With this product the sums form the group algebra $\mathbb{F}_2[G]$. The $|G| \times |G|$ binary matrices $\lambda(a)$ and $\rho(a)$ multiply by a on the left and on the right, respectively, and they differ only when G is non-abelian. Finally, the sum a^* replaces every element of a by its inverse (Appendix A).

The two-block family takes $G = Z_l \times Z_m$ and two sums of three distinct elements, $a = g_1 + g_2 + g_3$ and $b = h_1 + h_2 + h_3$. With $A = \lambda(a)$ and $B = \lambda(b)$, the code is

$$H_X = (A \mid B), \qquad H_Z = (B^{\mathsf{T}} \mid A^{\mathsf{T}}), \tag{4}$$

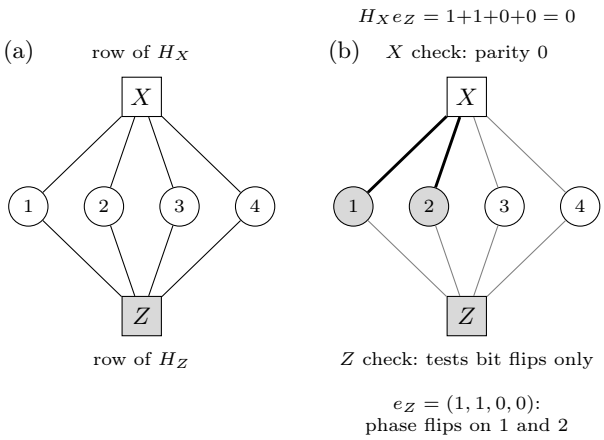


Fig. 1. (a) The Tanner graph of the $[[4, 2, 2]]$ code, $H_X = H_Z = (1\,1\,1\,1)$: qubits are circles, checks are squares and edges mark the 1s. (b) The X check misses $e_Z = (1, 1, 0, 0)$ (parity 0), and e_Z is not a sum of rows of H_Z. Because no single phase flip has both properties, $d_Z = 2$.

with $n = 2|G|$ physical qubits and checks of weight 6. We use $Z_6 \times Z_3$ ($n = 36$) and $Z_6 \times Z_6$ ($n = 72$). Bravyi et al. [5] call these bivariate bicycle codes, a family that includes the $[[144, 12, 12]]$ gross code, but they allow only elements $(i, 0)$ or $(0, j)$, whereas Eberhardt and Steffan [14] use the name for any a and b. Because we allow any (i, j), our family contains that of Bravyi et al., and we call it two-block codes over $Z_l \times Z_m$.

The lifted-product family is that of the mitten codes of Bhardwaj et al. [4] (their Definition 4), defined over a non-abelian G. Ten group elements and the identity 1 form four sums of three distinct elements,

$$\begin{aligned} a_1 &= g_1 + g_2 + g_3, \quad & a_2 &= 1 + g_4 + g_5, \\ b_1 &= h_1 + h_2 + h_3, \quad & b_2 &= 1 + h_4 + h_5. \end{aligned} \tag{5}$$

A base matrix has sums as its entries and becomes binary when each entry is replaced by its $|G| \times |G|$ matrix. The mitten codes are the lifted product [37,36] of the row $(a_1\ a_2)$ and the column $(b_1^*\ b_2^*)^{\mathsf{T}}$, and in our convention the row enters through λ and the column through ρ:

$$\begin{aligned} H_X &= \begin{pmatrix} \lambda(a_1) & 0 & \lambda(a_2) & 0 & \rho(b_1^*) \\ 0 & \lambda(a_1) & 0 & \lambda(a_2) & \rho(b_2^*) \end{pmatrix}, \\ H_Z &= \begin{pmatrix} \rho(b_1) & \rho(b_2) & 0 & 0 & \lambda(a_1^*) \\ 0 & 0 & \rho(b_1) & \rho(b_2) & \lambda(a_2^*) \end{pmatrix}, \end{aligned} \tag{6}$$

where 0 is the $|G| \times |G|$ zero matrix. The code has $n = 5|G|$ physical qubits, checks of weight 9 and, by Eq. (2), $k \geq |G|$. Bhardwaj et al. also require $\lambda(a_2)$ and $\rho(b_2)$ to be invertible, which gives $k = |G|$. We drop this requirement, so our

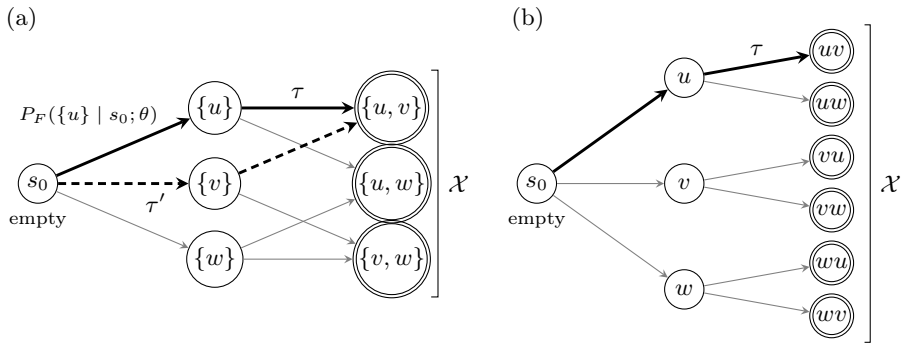


Fig. 2. (a) Objects as sets of two pieces from $\{u, v, w\}$. Complete objects ($\mathcal{X}$) have a double border, and $\{u, v\}$ has two trajectories, τ and τ'. (b) The same pieces in two ordered slots: each object has one trajectory, and the graph is a tree.

family is much larger (Appendix A.2). We use $Z_6 \times S_3$ ($n = 180$) and $Z_{10} \times S_3$ ($n = 300$), the latter being the group of their $[[300, 60, 14]]$ code.

The two-block spaces can be listed in full, which gives exact answer keys at 36 and 72 physical qubits (Sec. 4.3). We compare search methods on the lifted-product family, whose spaces are far too large to list. In this family d_X and d_Z can differ, so the best codes depend on the bias between bit and phase flips.

3.3 GFlowNets

A GFlowNet [1,2] builds an object one piece per step. A neural network with parameters θ, called the policy, chooses each piece and is trained to produce each complete object with probability proportional to a positive reward:

$$p_\theta(x) = \frac{R(x)}{Z}, \qquad Z = \sum_{x' \in \mathcal{X}} R(x'). \tag{7}$$

where $\mathcal{X}$ is the set of complete objects. When $\mathcal{X}$ is too large to list, the total reward Z cannot be computed, so it is learned as a scalar Z_θ alongside the network. This normaliser Z is unrelated to the phase flip Z and to the group Z_l.

The possible steps form a directed acyclic graph whose states are partial objects and whose actions each add one piece. A trajectory $\tau = (s_0 \to s_1 \to \cdots \to s_T)$ runs from the empty object s_0 to a complete object $s_T = x$ (Fig. 2), and the policy gives the probability $P_F(s' \mid s; \theta)$ of each step. When an object can be built in several orders, as $\{u, v\}$ can in Fig. 2(a), a backward policy $P_B(s \mid s')$ splits its reward among those orders. Trajectory balance [50] then requires every trajectory τ ending at x to satisfy

$$Z_\theta \prod_{t=0}^{T-1} P_F(s_{t+1} \mid s_t; \theta) = R(x) \prod_{t=0}^{T-1} P_B(s_t \mid s_{t+1}), \tag{8}$$

If Eq. (8) holds on every trajectory, then $Z_\theta = Z$ and p_θ satisfies Eq. (7).

4 Benchmark

4.1 Search spaces

A code is specified by the tuple $x = (g_1, g_2, g_3, h_1, h_2, h_3) \in G^6$ in the two-block family and by $x = (g_1, \ldots, g_5, h_1, \ldots, h_5) \in G^{10}$ in the lifted-product family. Because the elements within each sum are distinct, g_4, g_5, h_4 and h_5, which share a sum with the identity (Eq. (5)), are never the identity. Once each sum is translated to contain the identity, the elements must also generate G, that is, every element of G must be a product of them and their inverses. Otherwise the code splits into disconnected copies of a smaller code. In the lifted-product family this holds for each side separately: the elements of a_1 and a_2 must generate G, and so must those of b_1 and b_2. The answer keys at $n = 36$ and 72 do not apply this rule.

4.2 Equivalent codes and the canonical form

Two CSS codes are equivalent when relabelling the qubits and the checks turns the matrices of one into those of the other, and equivalent codes share n, k, d_X and d_Z. Five operations on a tuple produce an equivalent tuple (Appendix A.3). Among them are reordering the elements within a sum, translating a sum (multiplying all its elements by one group element), and conjugating the g_i alone or the h_i alone ($g_i \to g g_i g^{-1}$ for one g). The canonical form of a tuple undoes only reordering and translation. Two distinct canonical forms can still be equivalent codes, and a graph-isomorphism program decides equivalence exactly on the Tanner graphs. The mirror image of a code swaps H_X and H_Z, so that d_X and d_Z trade places, and in the lifted-product family we count a code and its mirror image as two codes.

4.3 Oracle and references

The oracle returns the score $s(x)$ of the code of a tuple x, with higher scores for better codes (this s is unrelated to the GFlowNet states). Every search method gets its scores only from the oracle, which first replaces the tuple by its canonical form. A form already scored in the run is answered from memory at no cost, whereas a new form is a paid call, so the cost of a method is the number of paid calls it makes (Table A.1 gives the cost of one). Each size has its own score. At 36 and 72 physical qubits every code's exact score is known, at 180 only codes near the top are checked exactly, and at 300 none are (Table 2). An answer key is a complete list of the codes of a family together with their exact distances.

36 and 72 physical qubits. Here the score is the merit of Ref. [12],

$$\mathrm{merit}(x) = \frac{k d^2}{n}, \qquad d = \min(d_X, d_Z). \tag{9}$$

The distance enters squared because a surface code, the standard planar code, needs about d^2 physical qubits per logical qubit to reach distance d. In this paper these keys grade a sampler (Appendix D.2), not a search.

Table 2. The answer keys and the reference sample at each size. The fifth column counts the canonical forms and the inequivalent codes (Sec. 4.2) with $k > 0$.

Size, family and group	Score	How a proposal is graded	Graded against	$k > 0$: forms / codes	Best or published codes
$n = 36$, two-block, $Z_6 \times Z_3$	merit, Eq. (9)	lookup	complete key: 2,304 forms, 48 inequivalent codes	704 / 20	$[[36, 4, 6]]$, merit 4.0: 48 forms, one code
$n = 72$, two-block, $Z_6 \times Z_6$	merit, Eq. (9)	lookup	complete key: 40,401 forms	9,280 / 55	$[[72, 12, 6]]$ of Bravyi et al. [5], merit 6.0: 48 forms, one code; $[[72, 4, 8]]$, largest d: 1,440 forms, 6 codes
$n = 180$, lifted product, $Z_6 \times S_3$	$s(x, \gamma)$, Eq. (10)	bound score; exact ladders at the threshold	reference sample: 666 uniform draws, exact distances	666 / 666	certified, $d_X = d_Z = 10$, from search runs
$n = 300$, lifted product, $Z_{10} \times S_3$	$s(x)$, Eq. (25)	Monte Carlo, BP-OSD decoder, 1,000 shots	reference rate from reference runs (Appendix E.1)	not computed	the mitten code $[[300, 60, 14]]$ [4]

180 physical qubits. The score at this size assumes that each qubit suffers a bit flip with probability p_X and, independently, a phase flip with probability p_Z. A code of phase-flip distance d_Z corrects every pattern of fewer than $d_Z/2$ phase flips, so the lightest pattern that defeats it has probability about $p_Z^{d_Z/2}$, and the same holds for bit flips with d_X and p_X. The score counts the orders of magnitude by which the more likely of these two failures is suppressed:

$$\begin{aligned} s(x, \gamma) &= \min(c_X d_X,\ c_Z d_Z), \\ c_X = \tfrac{1}{2} \log_{10} \frac{1}{p_X}, &\quad c_Z = \tfrac{1}{2} \log_{10} \frac{1}{p_Z}. \end{aligned} \tag{10}$$

A score of 10, for example, means that the lightest uncorrectable error has probability about 10^{-10}. The noise bias $\gamma = \log_{10}(p_X/p_Z)$ says which flip is more likely, while the rarer flip is fixed at 10^{-3}. We report results at $\gamma = \pm 1$, where the likelier flip has probability 10^{-2} and coefficient 1.0, and the rarer one has coefficient 1.5.

Proposals are graded against the uniform reference sample, which consists of the 666 of 1,000 uniform draws whose elements generate G, each with exact distances. Its highest score at $\gamma = \pm 1$ is 10.0, reached by 6 of the 666, and this is the threshold a top-scoring code must reach. At $\gamma = 0$, by contrast, 116 of the 666 share the sample's highest score, so an unbiased search has little to find.

Exact distances at this size come from what we call the SAT ladder, an established use of a satisfiability solver [9]. For $w = 1, 2, \ldots$ in turn, the ladder asks whether a logical operator of weight at most w exists, and a question left undecided within its budget stops the ladder at a lower bound. Two such ladders, the exact ladders, give d_X and d_Z. They are expensive, so every new code first gets a cheap and slightly optimistic bound score, computed from the distances of two classical codes built from its blocks, with those distances found by classical ladders (Appendix A). At $\gamma = \pm 1$, the same paid call also runs the exact ladders whenever the bound score reaches the threshold or a classical ladder is left un-

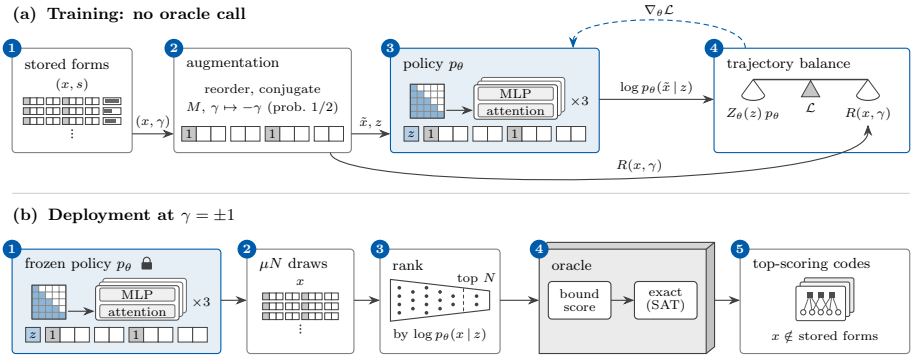


Fig. 3. GFlowQEC-offline ($n = 180$). Rows of slots are tuples, with grey slots fixed to the identity, and Tanner graphs are codes. (a) Training, with no oracle call: (1) a stored form with a bias γ and its reward $R(x, \gamma)$, (2) augmented and, with probability 1/2, mirrored by M with $\gamma \mapsto -\gamma$, (3) read by the transformer (three blocks, the shaded triangle a causal mask) with z on its start token, (4) the balance, the trajectory-balance loss weighing $R(x, \gamma)$ against $Z_\theta(z)\, p_\theta$, and its gradient step (dashed). (b) Deployment: (1) the frozen policy at $\gamma = \pm 1$, (2) μN drawn forms, (3) the N with the highest $\log p_\theta(x \mid z)$, (4) the oracle's bound score and SAT certification, (5) the top-scoring codes outside the stored forms.

decided. A code on which they have run is certified, and it counts as top-scoring when the score they prove reaches the threshold. A ladder that stops early, as happens for 0.7 per cent of certified codes, or a classical bound that fails can leave a good code uncounted, but neither can make a bad code count.

5 GFlowQEC

GFlowQEC's policy is a causal transformer [48] that builds the tuple x one position, or slot, at a time, and it is trained so that, at the optimum,

$$p_\theta(x) \propto R(x) = \max(s(x), 0.01)^\beta, \tag{11}$$

where $p_\theta(x)$ is the probability that the policy builds x, the score $s(x)$ is clamped at 0.01, and the exponent $\beta > 0$ sets how strongly the policy favours high scores. We use it at 180 physical qubits, over $Z_6 \times S_3$, where it is trained once on stored scores and then frozen (Fig. 3).

5.1 Slot-by-slot construction

The lifted-product tuple has $T = 10$ slots, which the policy fills in the order $g_1, \ldots, g_5, h_1, \ldots, h_5$. We write x_t for the element in slot t, $x_{<t}$ for the elements before it, and $P_F(x_t \mid x_{<t}; \theta)$ for the policy's probability of x_t. Slots g_1 and h_1 are fixed to the identity. By translation this loses no code up to equivalence, and

it puts the identity in every sum. To keep the elements of each sum distinct, a mask restricts every other slot t to its legal set, which is the group without the identity and without the elements already placed in the same sum,

$$\mathcal{A}_t(x_{<t}) = G \setminus \big(\{1\} \cup \{x_{t'} : t' < t, \\ t' \text{ in the same sum as } t\}\big). \tag{12}$$

A tuple is legal when $g_1 = h_1 = 1$ and every other element lies in its legal set. A cheap test, the generation check, decides without an oracle call whether the elements generate G, and a tuple that fails it is never sent to the oracle.

5.2 Objective and the choice of β

Because a partly filled tuple can be reached only by filling its slots in order, the state graph is a tree, as in Fig. 2(b), and $P_B = 1$. The probability of producing x is then the product of the slot probabilities,

$$p_\theta(x) = \prod_{t=1}^{T} P_F(x_t \mid x_{<t}; \theta). \tag{13}$$

Trajectory balance (Eq. (8)) therefore reduces to $Z_\theta\, p_\theta(x) = R(x)$, and we minimise the squared gap between the logarithms of its two sides,

$$\mathcal{L}(x) = \Big(\log Z_\theta + \sum_{t=1}^{T} \log P_F(x_t \mid x_{<t}; \theta) \\ - \log R(x)\Big)^2, \tag{14}$$

with log the natural logarithm. If training reaches the optimum of this loss, Eq. (11) holds exactly on the legal tuples that pass the generation check.

We set the exponent to $\beta = 8$ before training, by a fixed rule applied to the uniform reference sample (Appendix B).

5.3 Training data

The policy trains only on scores that 42 earlier runs of several search methods had already paid for, so training itself makes no oracle call. Those runs made 126,000 paid calls, which cover 116,972 distinct canonical forms, the stored forms. Each form is stored with the two distances the oracle returned, exact for certified codes and bounds otherwise (Appendix B.1), and these give its score at any bias (Eq. (10)).

During training, each drawn tuple is replaced by a random equivalent tuple $\tilde{x}$ with the same reward, obtained by reordering and conjugation (Appendix B). A single network serves both biases by taking the bias as an input, a condition vector z, and each bias has its own learned normaliser $Z_\theta(z)$. With probability one half, a training tuple is also replaced by its mirror image at the opposite bias, which has the same score. We train one model per random seed, ten in all, at $\gamma = \pm 1$, the biases at which they are deployed, and then freeze them. Table B.1 lists every setting.

5.4 Deployment

The frozen policy, GFlowQEC-offline, proposes codes at $\gamma = +1$ or $\gamma = -1$ (the online variant, GFlowQEC-online, which keeps learning inside a local search, is described in Appendix E). It draws until it holds μN distinct canonical forms, where $\mu = 10$ or 100 is an oversample factor, and discards without an oracle call every draw that fails the generation check or repeats a form. It then ranks the forms by $\log p_\theta(x \mid z)$, the log-probability of the tuple as drawn, which stands in for the score (Appendix B.1), and pays one oracle call for each of the $N = 1{,}000$ highest. We chose this ranking after earlier runs had been read, because an earlier deployment that paid for its draws in the order drawn had trailed an earlier, smaller supervised screen (Appendix D).

Training fixes $p_\theta(x \mid z)$ only on the stored tuples and their augmented and mirrored copies, and the augmentation reaches only 16 of the up to 1,296 legal tuples of a canonical form. Everywhere else the ranking is the network's extrapolation, so we use the policy as a proposer that ranks its own draws, not as a calibrated sampler.

6 Experimental setup

All comparisons are made at $n = 180$, the size at which the stored scores exist and top-scoring codes can be certified exactly. Every run is graded on its first 1,000 paid calls, at $\gamma = +1$ and at $\gamma = -1$.

6.1 Comparison methods

Every method uses the same oracle, search space and budget, and runs are paired by random seed. Table C.1 gives each method as run.

- *Tuned parallel tempering* (tuned PT) [46,16,19] runs eight chains of local search at different temperatures, with Metropolis acceptance [32] and exchanges between the chains. It uses the temperature range that did best on its own runs, a choice that favours it. A variant starts its chains at stored forms rather than at uniform draws, and those starting forms never count toward its results.
- The *supervised screen* uses the policy's network as a regressor that predicts a code's score. Unlike the policy, it sees all ten slots at once, and it also reads relational counts, which do not change when a sum is translated (Appendix B.2). It is trained by squared error to predict the scores of the same stored forms, with the policy's augmentation, batches and seeds. Its variant and training length were tuned on a held-out tenth of the stored forms, a choice the policy did not get, after which it was trained on all of them. When deployed, it ranks ten or a hundred times N uniform random candidates and pays for its top N.

- *Soft Q-learning* [18] trained offline uses the same stored scores, network, training steps and deployment rule as GFlowQEC-offline. Because on our construction it has the same optimal policy as the GFlowNet [47], it tests whether the training objective matters.
- *MAP-Elites* [33] keeps the best code found in each bin of a grid of cheap code properties, and we tuned it over two bin grids.
- *Bayesian optimisation* (BO) fits a Gaussian process to a one-hot encoding of the tuple. It is not the method of Chengyu et al. [10], who use a learned graph embedding and a different decoder, noise rate and code family.
- *Uniform sampling* pays for uniform random tuples that pass the generation check.

We also ran PPO [42] at two entropy weights, MAP-Elites with 20 bins, and soft Q-learning and Munchausen DQN [49] trained during the search, and Appendix D reports them.

6.2 Measures

The main measure is the number of certified top-scoring codes in a run's first 1,000 paid calls. A paid code is novel when its canonical form is not among the stored forms. For a method trained on or started from the stored forms only novel codes count, whereas the other methods never saw the stored forms, so all their codes count. Because a novel code can still be equivalent to a stored code or its mirror image, Sec. 7 also counts inequivalent codes.

To compare two methods we use their pooled ratio, which divides their counts, each summed over the seeds both methods ran. We say that the two are not separated when the ratio's 95 per cent interval (Appendix C.2) contains 1.

7 Results

Table 3 gives the counts, with GFlowQEC-offline and the supervised screen at $\mu = 10$. In every paired seed and at both biases, GFlowQEC-offline (below, the policy) found more top-scoring codes than every other method in the table except soft Q-learning trained offline. Each pair of numbers below is at $\gamma = +1$ and $\gamma = -1$, in that order.

Against tuned PT. Counting inequivalent codes, tuned PT is the strongest method that does not use the stored data, although MAP-Elites with five bins found slightly more top-scoring codes before equivalent ones were merged. Counting only inequivalent codes, the policy found 2.4 and 2.8 times as many top-scoring codes as tuned PT, and more than twice as many in 19 of its 20 runs. Starting tuned PT from the stored forms does not close the gap, since on the same count the policy still found 2.8 and 2.7 times as many.

Table 3. Results at 180 physical qubits: certified top-scoring codes in each run's first 1,000 paid calls, mean over ten seeds rounded to whole codes, at noise bias $\gamma = +1$ and $\gamma = -1$ (per seed in Table D.1). In the first group of rows only novel codes count. Inequivalent drops novel codes equivalent to a stored code or its mirror image and counts each equivalence class once (Sec. 6.2). Ratios in the text pool the seeds both methods ran.

	Certified top-scoring codes		Inequivalent	
Method	$\gamma = +1$	$\gamma = -1$	$\gamma = +1$	$\gamma = -1$
Trained on, or started from, the stored evaluations				
GFlowQEC-offline	313	317	221	222
Supervised screen	87	84	62	59
Soft Q-learning, trained offline	317	364	225	259
Tuned PT from the stored forms	119	123	79	84
Searching without stored data				
Tuned PT	97	84	92	79
MAP-Elites, five bins	103	92	88	74
Uniform sampling	12	8	12	8
BO, one-hot encoding	50	53	49	52

Against soft Q-learning. Soft Q-learning trained offline was not separated from GFlowQEC-offline at $\gamma = +1$ and found more at $\gamma = -1$. It found 1.01 and 1.15 times the policy's count, and at $\gamma = -1$ the interval, $[1.105, 1.19]$, excludes 1.

Against the supervised screen. At $\mu = 10$ the policy found 3.6 and 3.8 times as many top-scoring codes as the screen, and led in each of ten seeds. When both oversampled at $\mu = 100$, the screen found more in each of ten seeds, and the policy found 0.45 and 0.62 times its count. Still, two controls show that the policy's strength lies in its draws. Ranking the screen's uniform candidates instead of its own draws, the policy found a fifth as many codes as from its own draws, and fewer than the screen. Ranking the policy's draws instead of uniform candidates at $\mu = 100$, the screen found 1.27 and 1.30 times its own count, more in every seed, so better draws and better ranking combine.

Novelty. We group top-scoring codes into neighbourhoods, codes whose distance-limiting half of the tuple agrees up to conjugation (Appendix C). In each of the policy's runs, 87 to 93 per cent of its neighbourhoods also hold a stored top-scoring code, so the policy mostly finds new partners for good halves that the stored data already held.

Cost. The policy's paid calls ran the exact ladders 3.5 and 4.2 times as often as tuned PT's, and these ladders dominate the cost of a paid call here. On seeds 0 to 2, per CPU second of exact ladders, the policy and tuned PT found top-scoring codes at the same rate, within five per cent at both biases, and on the

inequivalent counts tuned PT was ahead. At seeds 3 to 9 the two ran on different machines, so we do not compare their CPU seconds. We keep the paid call as the unit because on a device every candidate costs one circuit-level simulation, whatever proposed it.

8 Discussion

A trained proposer is most likely to pay off when four conditions hold together: good codes are too rare to find by sampling, no quantity that is cheap to compute from the parity checks ranks them, each call to the oracle is expensive, and the cost of training is shared across many searches. Of these, only the cost of a call is known to grow, both with the size of the code and under circuit-level noise. And searches already run at 600 to 1,500 physical qubits [29], past the few hundred physical qubits where exhaustive catalogues stop [27].

Limitations.

- The gain is in how many good codes a budget buys, not in the best code found. We show no code better than those found by Bayesian optimisation, reinforcement learning or LLM-driven searches. Nor does a good code at this size need a trained policy, since uniform sampling found about ten top-scoring codes per 1,000 paid calls (Table 3).
- The lead is counted per paid call, with nothing charged for the stored data or the policy's training, and it disappears per CPU second of exact ladders (Sec. 7).
- The score at 180 physical qubits is computed from distances alone (Eq. (10)), with no decoder. We have not measured whether circuit-level noise, with faulty check measurements and a real decoder, reorders the codes.
- Proportional sampling was graded only on an earlier, simpler sampler, at $n = 36$ and 72, where the exact target is known (Appendix D.2), and not on the policy evaluated in this paper.
- In early test runs at 180 physical qubits, GFlowQEC-online (Appendix E) found far fewer top-scoring codes than tuned PT. At 300 physical qubits more of its proposals than tuned PT's reached a reference error rate, but fewer did when near-duplicates were counted only once.

9 Conclusion

Trained once on stored scores and used frozen at 180 physical qubits, GFlowQEC found more than twice as many top-scoring codes per paid call as tuned PT, counting only codes inequivalent to its training data. It found more than three and a half times as many as a supervised screen that ranked ten random candidates per call, although a screen that ranked a hundred found more. Any search can be graded on our benchmark, on the exact answer keys at 36 and 72 physical qubits and against the reference sample at 180. Our software and data

are public, with the code at https://github.com/NunoMBatista/GFlowQEC (archived at https://doi.org/10.5281/zenodo.23922222) and the data at https://huggingface.co/datasets/NunoBatista/gflowqec-data.

Future work. The next step toward choosing a code for a device is to re-score the top-scoring codes at 180 physical qubits under circuit-level noise and see whether their ranking survives. The proposer would then be graded at that cost, against the method of Chengyu et al. [10] at the budget that method is built for, a few hundred calls to an expensive oracle. Because device noise is modelled only approximately and drifts between calibrations, a varied set of good codes can be worth more than a single best one. A policy conditioned on the measured noise through multi-objective GFlowNets [20] could then propose a new set after each calibration, without running a new search.

The setting we aim for is a quantum compiler that chooses the code itself. A strategic plan for neutral-atom quantum computing [31] calls for the compiler to select codes from the application and the hardware specification, and names generative neural networks among the methods that could find codes suited to a device's noise and operational constraints. GFlowQEC fits this role as a conditional generator, in which one policy would answer every request. The hard constraints of a request, such as the device's connectivity, the check weight or a floor on k, would be built into the mask of Eq. (12), and its noise model would enter through the condition z. Because the policy would learn from every paid call, later requests would need fewer of them.

A single policy could also search across code families if each action chose one level of a construction in turn: the family, the group, the base matrices and the sums that fill them. Liu and Marquardt [29] let a language model mutate a three-level grammar of this kind, whereas ours would be one trained policy choosing every level. Finally, each method here has a setting that trades how many good codes it finds against how varied they are (β for the policy, the temperatures for parallel tempering and the entropy weight for PPO), and a fair comparison would sweep each setting and compare the resulting fronts.

Acknowledgments. The authors had most of the ideas, designed and directed the experiments and carried out the literature review. We used the large language models Claude Fable 5.1, Claude Opus 5.5 and Claude Sonnet 5.5 (Anthropic) in this work. They wrote almost all of the code and helped summarise the literature, analyse the results, and write and edit the text, always under the authors' guidance. The authors reviewed the text and take full responsibility for its content.

References

1. Bengio, E., Jain, M., Korablyov, M., Precup, D., Bengio, Y.: Flow network based generative models for non-iterative diverse candidate generation. In: NeurIPS. vol. 34, pp. 27381–27394 (2021)
2. Bengio, Y., Lahlou, S., Deleu, T., Hu, E.J., Tiwari, M., Bengio, E.: GFlowNet foundations. Journal of Machine Learning Research **24**(210), 1–55 (2023)

3. Berthusen, N., Devulapalli, D., Schoute, E., Childs, A.M., Gullans, M.J., Gorshkov, A.V., Gottesman, D.: Toward a 2D local implementation of quantum low-density parity-check codes. PRX Quantum **6**, 010306 (2025). doi:10.1103/PRXQuantum.6.010306
4. Bhardwaj, A., Ma, M., Meister, N., King, R., Bluvstein, D., Preskill, J., Cain, M., Xu, Q., Huang, H.Y.: High-rate qLDPC processors. arXiv:2607.28795 (2026)
5. Bravyi, S., Cross, A.W., Gambetta, J.M., Maslov, D., Rall, P., Yoder, T.J.: High-threshold and low-overhead fault-tolerant quantum memory. Nature **627**(8005), 778–782 (2024). doi:10.1038/s41586-024-07107-7
6. Breuckmann, N.P., Eberhardt, J.N.: Quantum low-density parity-check codes. PRX Quantum **2**(4), 040101 (2021). doi:10.1103/PRXQuantum.2.040101
7. Calderbank, A.R., Shor, P.W.: Good quantum error-correcting codes exist. Phys. Rev. A **54**(2), 1098–1105 (1996). doi:10.1103/PhysRevA.54.1098
8. Chen, R., Wang, X., Hu, R., Li, Z., Huang, L.: Beyond the proxy: Trajectory-distilled guidance for offline GFlowNet training. In: ICML (2026)
9. Chen, Y.F., Jafari, S.M.R., Lai, C.Y.: SAT, MaxSAT, and SMT for QLDPC Distance Computation: A Large-Scale Empirical Study. arXiv:2606.12445 (2026)
10. Chengyu, Y., Meister, R., Carty, C., Lin, S.K., Bondesan, R.: Bayesian optimization for quantum error-correcting code discovery. arXiv:2601.18562 (2026)
11. Cruz-Benito, J.: Large Language Model-Guided Discovery of Weight-Five Bivariate Bicycle Codes. arXiv:2610.06623 (2026)
12. Cruz-Benito, J., Cross, A.W., Kremer, D., Faro, I.: Evolutionary discovery of bivariate bicycle codes with LLM-guided search. arXiv:2606.02418 (2026)
13. Dai, J., Rizvi-Martel, M., Rabusseau, G.: FlowQ-Net: A Generative Framework for Automated Quantum Circuit Design. arXiv:2510.26688 (2025)
14. Eberhardt, J.N., Steffan, V.: Logical operators and fold-transversal gates of bivariate bicycle codes. IEEE Trans. Inf. Theory **71**(2), 1140–1152 (2025). doi:10.1109/TIT.2024.3521638
15. Freire, B.C.A., Delfosse, N., Leverrier, A.: Optimizing hypergraph product codes with random walks, simulated annealing and reinforcement learning. arXiv:2501.09622 (2025), IEEE ISIT 2025
16. Geyer, C.J.: Markov chain Monte Carlo maximum likelihood. In: Computing Science and Statistics: 23rd Symposium on the Interface. pp. 156–163 (1991)
17. Glasserman, P., Yao, D.D.: Some guidelines and guarantees for common random numbers. Management Science **38**, 884 (1992). doi:10.1287/mnsc.38.6.884
18. Haarnoja, T., Tang, H., Abbeel, P., Levine, S.: Reinforcement learning with deep energy-based policies. In: ICML (2017)
19. Hukushima, K., Nemoto, K.: Exchange Monte Carlo method and application to spin glass simulations. J. Phys. Soc. Jpn. **65**, 1604 (1996). doi:10.1143/JPSJ.65.1604
20. Jain, M., Raparthy, S.C., Hernandez-Garcia, A., Rector-Brooks, J., Bengio, Y., Miret, S., Bengio, E.: Multi-Objective GFlowNets. In: ICML (2023)
21. Jones, D.R., Schonlau, M., Welch, W.J.: Efficient global optimization of expensive black-box functions. J. Global Optim. **13**, 455 (1998). doi:10.1023/A:1008306431147
22. Katz, D., Baptista, J., Azen, S.P., Pike, M.C.: Obtaining confidence intervals for the risk ratio in cohort studies. Biometrics **34**, 469 (1978)
23. Kim, M., Ko, J., Yun, T., Zhang, D., Pan, L., Kim, W., Park, J., Bengio, E., Bengio, Y.: Learning to Scale Logits for Temperature-Conditional GFlowNets. In: ICML (2024)

24. Kim, M., Yun, T., Bengio, E., Zhang, D., Bengio, Y., Ahn, S., Park, J.: Local Search GFlowNets. In: ICLR (2024)
25. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. In: ICLR (2015)
26. Koo, I., Cha, H., Lee, J.: QFlowNet: Fast, diverse, and efficient unitary synthesis with generative flow networks. In: IEEE QCNC 2026 (2026)
27. Liang, Z., Liu, K., Song, H., Chen, Y.A.: Generalized toric codes on twisted tori for quantum error correction. PRX Quantum **6**, 020357 (2025). doi:10.1103/rmy6-9n89
28. Lin, H.K., Pryadko, L.P.: Quantum two-block group algebra codes. Phys. Rev. A **109**(2), 022407 (2024). doi:10.1103/PhysRevA.109.022407
29. Liu, Z., Marquardt, F.: Large-Language-Model Discovery of Quantum LDPC Codes through Structured Concept Evolution. arXiv:2606.24808 (2026)
30. MacKay, D.J.C., Mitchison, G., McFadden, P.L.: Sparse-graph codes for quantum error correction. IEEE Trans. Inf. Theory **50**(10), 2315–2330 (2004). doi:10.1109/TIT.2004.834737
31. Menssen, A., Wang, T.T., Gullans, M., et al.: Strategic Plan for Neutral Atom Quantum Computation. arXiv:2607.21554 (2026)
32. Metropolis, N., Rosenbluth, A.W., Rosenbluth, M.N., Teller, A.H., Teller, E.: Equation of state calculations by fast computing machines. J. Chem. Phys. **21**, 1087 (1953). doi:10.1063/1.1699114
33. Mouret, J.B., Clune, J.: Illuminating search spaces by mapping elites. arXiv:1504.04909 (2015)
34. Olle, J., Zen, R., Puviani, M., Marquardt, F.: Simultaneous Discovery of Quantum Error Correction Codes and Encoders with a Noise-Aware Reinforcement Learning Agent. npj Quantum Information (2024). doi:10.1038/s41534-024-00920-y
35. Panteleev, P., Kalachev, G.: Degenerate quantum LDPC codes with good finite length performance. Quantum **5**, 585 (2021). doi:10.22331/q-2021-11-22-585
36. Panteleev, P., Kalachev, G.: Asymptotically good quantum and locally testable classical LDPC codes. In: STOC 2022. pp. 375–388 (2022). doi:10.1145/3519935.3520017
37. Panteleev, P., Kalachev, G.: Quantum LDPC codes with almost linear minimum distance. IEEE Trans. Inf. Theory **68**(1), 213–229 (2022). doi:10.1109/TIT.2021.3119384
38. Qian, D., Li, T.: Multi-agent discovery of practical quantum LDPC codes. arXiv:2608.08996 (2026)
39. Roffe, J.: LDPC: Python tools for low density parity check codes. https://pypi.org/project/ldpc/ (2022)
40. Roffe, J., White, D.R., Burton, S., Campbell, E.T.: Decoding across the quantum low-density parity-check code landscape. Phys. Rev. Research **2**(4), 043423 (2020). doi:10.1103/PhysRevResearch.2.043423
41. Schulman, J., Moritz, P., Levine, S., Jordan, M.I., Abbeel, P.: High-dimensional continuous control using generalized advantage estimation. In: ICLR (2016)
42. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. arXiv:1707.06347 (2017)
43. Sella, O.S., Pinsler, R., Heinis, T.: Designing Quantum Error Correcting Codes to fit decoders via Reinforcement Learning. arXiv:2608.15754 (2026)
44. Steane, A.M.: Multiple-particle interference and quantum error correction. Proc. R. Soc. London, Ser. A **452**(1954), 2551–2577 (1996). doi:10.1098/rspa.1996.0136

45. Su, V.P., Cao, C., Hu, H.Y., Yanay, Y., Tahan, C., Swingle, B.: Discovery of Optimal Quantum Error Correcting Codes via Reinforcement Learning. Phys. Rev. Applied **23**, 034048 (2025). doi:10.1103/PhysRevApplied.23.034048
46. Swendsen, R.H., Wang, J.S.: Replica Monte Carlo simulation of spin-glasses. Phys. Rev. Lett. **57**, 2607 (1986). doi:10.1103/PhysRevLett.57.2607
47. Tiapkin, D., Morozov, N., Naumov, A., Vetrov, D.: Generative Flow Networks as Entropy-Regularized RL. In: AISTATS (2024)
48. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: NeurIPS (2017)
49. Vieillard, N., Pietquin, O., Geist, M.: Munchausen reinforcement learning. In: NeurIPS (2020)
50. Whitammer, E.S., Jain, M., Bengio, E., Sun, C., Bengio, Y.: Trajectory balance: Improved credit assignment in GFlowNets. In: NeurIPS. vol. 35, pp. 5955–5967 (2022). doi:10.52202/068431-0431
51. Wilson, E.B.: Probable inference, the law of succession, and statistical inference. J. Am. Stat. Assoc. **22**, 209 (1927). doi:10.1080/01621459.1927.10502953
52. Yan, G., Li, S., Ma, P., Zhang, Q., Ma, P., Wang, J., Hsieh, M.H., Du, Y.: OmniQEC: discovering practical quantum error-correcting codes by an AI scientist. arXiv:2607.25865 (2026)

A Codes and benchmark

We use the following groups.

- Z_l is the integers mod l under addition, and $Z_l \times Z_m$ is the set of pairs (i, j), added one slot at a time. In $Z_6 \times Z_3$, for example, $(4, 2) + (5, 2) = (3, 1)$.
- S_3 is the six orderings of three objects, and two orderings combine by applying one after the other. Starting from the order $(0, 1, 2)$, swapping the first two objects and then moving every object one place to the right gives $(2, 1, 0)$, whereas doing the two steps in the other order gives $(0, 2, 1)$. In S_3, then, the order of combination matters.
- For any two groups G_1 and G_2, $G_1 \times G_2$ is the set of pairs (g_1, g_2), combined one slot at a time. Writing $|G|$ for the number of elements of a group G, the product has $|G_1| \cdot |G_2|$ elements, so $Z_6 \times S_3$, for example, has 36.

A group is abelian when combining two elements gives the same result in either order. The group $Z_l \times Z_m$ is abelian, while S_3 and $Z_l \times S_3$ are not and are called non-abelian.

Let G be any finite group, in this paper $Z_l \times Z_m$ or $Z_l \times S_3$. A sum such as $a = g_1 + g_2 + g_3$ is a set of elements of G written with plus signs, and the plus signs say how two sums multiply: every element of one is multiplied by every element of the other, and the products are added modulo 2, so a product that appears twice cancels. In an abelian group, for example, $(g_1 + g_2)^2 = g_1^2 + g_1 g_2 + g_2 g_1 + g_2^2 = g_1^2 + g_2^2$. The set of all such sums, $\mathbb{F}_2[G]$, is called the group algebra of G. If a sum is written as a binary vector with one entry per element of G, multiplying it by a fixed sum a becomes multiplication by a matrix. Column v of the $|G| \times |G|$ matrix $\lambda(a)$ marks the elements gv for every g in a, so $\lambda(a)$ multiplies by a on the left. The matrix $\rho(a)$, whose column v marks the elements vg, multiplies on the right. With $G = Z_3$ and $a = 0 + 1$, for example, column v of $\lambda(a)$ marks the elements $0 + v$ and $1 + v$:

$$\lambda(a) = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix}.$$

The matrix $\rho(a)$ marks $v + 0$ and $v + 1$ instead. For an abelian group this is the same matrix and for S_3 a different one, which is why the lifted-product family needs both. The sum a^* replaces every element by its inverse, here $a^* = 0 + 2$, and $\lambda(a^*)$ is the transpose of $\lambda(a)$.

For an element $g \in G$, $\lambda(g)$ and $\rho(g)$ are the $|G| \times |G|$ permutation matrices with entries

$$\begin{aligned} \lambda(g)_{u,v} &= \begin{cases} 1 & \text{if } u = gv, \\ 0 & \text{otherwise,} \end{cases} \\ \rho(g)_{u,v} &= \begin{cases} 1 & \text{if } u = vg, \\ 0 & \text{otherwise.} \end{cases} \end{aligned} \tag{15}$$

For a sum a, $\lambda(a)$ and $\rho(a)$ are the sums mod 2 of $\lambda(g)$ and $\rho(g)$ over its elements. We rely on three facts about these matrices.

(i) $\lambda(g)\lambda(h) = \lambda(gh)$ and $\rho(g)\rho(h) = \rho(hg)$.
(ii) $\lambda(g)^{\mathsf{T}} = \lambda(g^{-1})$ and $\rho(g)^{\mathsf{T}} = \rho(g^{-1})$, so $\lambda(a)^{\mathsf{T}} = \lambda(a^*)$ and $\rho(a)^{\mathsf{T}} = \rho(a^*)$.
(iii) $\lambda(g)\rho(h) = \rho(h)\lambda(g)$ for all g, h: both move position u to guh, even in a non-abelian group.

A.1 Two-block family

Every choice of a and b satisfies the condition of Eq. (1), because $H_X H_Z^{\mathsf{T}} = AB + BA$ and $AB = BA$ in an abelian group [5,28]. Each such code is also equivalent to its mirror image (operation (f) of Appendix A.3) [28], so $d_X = d_Z$.

A.2 Lifted-product family

The two-block code of Eq. (4) is the lifted product of the 1×1 base matrices (a) and (b), the simplest case of Eq. (6). The lifted product of Eq. (6), which is Eq. (12) of Ref. [37], satisfies Eq. (1) for any group [37,36]. This holds because every $\lambda(a_j)$ commutes with every $\rho(b_i^*)$ by fact (iii), and the lifted product of two element-wise commuting matrices is a CSS code [37]. Over $Z_6 \times S_3$, $\lambda(1+g_4+g_5)$ is invertible for only 123 of the 595 pairs of distinct elements other than the identity, so the invertibility required by Bhardwaj et al. excludes most choices of a_2, and likewise of b_2. Without that requirement, our family also contains codes with $k > |G|$.

A.3 Relabellings behind Sec. 4.2

We use six operations on a tuple. Operations (a) to (e) give an equivalent tuple, and (f) gives the tuple of the mirror image.

(a) Reordering the elements within a sum.
(b) Translation, which multiplies all the elements of a sum by one group element.
(c) Renaming by an automorphism, a one-to-one renaming of the elements of G that respects the multiplication table.
(d) Conjugation of the g_i alone, $g_i \to g g_i g^{-1}$ for one g, or of the h_i alone.
(e) Swapping a and b, in the two-block family.
(f) The mirror image, which replaces each a_i by b_i^* and each b_i by a_i^*.

To find the canonical form of a tuple, we translate each sum so that each of its three elements in turn becomes the identity, write each result as a sorted list of element numbers and keep the smallest list. The items below take the operations in turn. Each concerns the lifted-product family except (e), and each was checked as an exact matrix equality on one random tuple per family.

(a) Reordering a sum leaves it unchanged.
(b) By fact (i), $\lambda(a_1 g) = \lambda(a_1)\lambda(g)$, so relabelling the qubits of the block columns that hold $\lambda(a_1)$, and the Z checks of the first block row (fact (iii)), absorbs $\lambda(g)$. The other sums work the same way.
(c) $\lambda(\varphi(g))$ is $\lambda(g)$ with every index v renamed $\varphi(v)$, and likewise for ρ.

(d) By facts (i) to (iii), relabelling every block by $\lambda(g)$ conjugates the blocks of the g_i and leaves those of the h_i unchanged. Relabelling by $\rho(h)^{\mathsf{T}}$ does the same for the h_i.
(e) Exchanging the two blocks of qubits undoes the swap of a and b.
(f) Renaming every index v as v^{-1} turns $\lambda(g)$ into $\rho(g^{-1})$ and $\rho(g)$ into $\lambda(g^{-1})$. Combined with an exchange of the second and third blocks of qubits, this renaming carries H_Z to the new H_X and H_X to the new H_Z.

Bhardwaj et al. [4] write the right action as $u = vg^{-1}$, so a tuple of their Definition 4 enters Eq. (6) with each h_i inverted. The $[[300, 60, 14]]$ entry of their Table XIII is already in our convention.

A.4 Search spaces, oracle and distances

If the elements of a tuple, with each sum translated to contain the identity, generate only a proper subgroup K of G, the tuple builds $|G|/|K|$ disconnected copies of a code over K (Sec. IV C of Ref. [28]).

Table A.1. The oracle at each size and the cost of one paid call on one CPU core. At $n = 180$ the median and mean are over the 126,000 bound scores of the 42 stored runs.

n, group	Score	Instrument	Cost per paid call
36, $Z_6 \times Z_3$	merit	exact d by enumeration	below 0.1 s
72, $Z_6 \times Z_6$	merit	exact d: exhaustive search, decoder-based upper bound [5], SAT ladder	0.15 s median, 12 s mean
180, $Z_6 \times S_3$	Eq. (10)	bounds d_A, d_B; exact d_X, d_Z by SAT ladders if certified	0.8 s median, 1.5 s mean; exact 10 s (uniform draw), about 2 min (certified)
300, $Z_{10} \times S_3$	Eq. (25)	BP-OSD, $p = 0.07$, 1,000 shots	1 to 2 s

We run BP-OSD in its combination-sweep variant of order 7 [40], with normalised min-sum belief propagation at scaling factor 0.625, a serial schedule, at most 100 iterations and prior $2p/3$ [39].

The bound score uses d_A and d_B, the distances of the classical codes with check matrices $A = (\lambda(a_1) \mid \lambda(a_2))$ and $B = (\rho(b_1) \mid \rho(b_2))$. These are the blocks of Eq. (6), not the A, B of Eq. (4). SAT ladders compute these distances too,

and we call them the classical ladders. A codeword of A, that is, a binary vector v with $Av = 0$, placed on two of the five blocks of qubits, is a phase-flip pattern that no X check detects. Unless the lightest such codeword is a stabilizer, it is a logical operator, and then $d_Z \le d_A$. In the same way, $d_X \le d_B$ unless the lightest codeword of B is a stabilizer.

Theorem 8 of Ref. [4] proves $d_Z \le d_A$ and $d_X \le d_B$ whenever A and B have full row rank, which is the case for 383 of the 666 codes of the reference sample. Both inequalities held on all 666.

B Policy and training

Slots and the normaliser. For every $g \in G$ the network outputs a logit $\ell_\theta(g; x_{<t})$, an unnormalised log-probability, and the policy is the softmax of these logits over the legal set,

$$P_F(x_t = g \mid x_{<t}; \theta) = \frac{\exp \ell_\theta(g; x_{<t})}{\sum_{g' \in \mathcal{A}_t(x_{<t})} \exp \ell_\theta(g'; x_{<t})} \tag{16}$$

for $g \in \mathcal{A}_t(x_{<t})$, and zero otherwise. At a fixed slot, $P_F = 1$ on the identity. Because the legal set has $|G| - 1$ elements at the first free slot of a sum and $|G| - 2$ at the second, the lifted-product family has $|\mathcal{X}| = ((|G| - 1)(|G| - 2))^4$ legal tuples.

At the optimum, a canonical form has probability proportional to its reward times its number of legal tuples. Each sum contributes a factor of six (three translations that put the identity into the sum, times two orders of its other two elements), which gives 1,296 in the lifted-product family. A coset sum, that is, a translate of a subgroup of order 3, gives the same set under all three of its translations, so its form gets less probability.

An online run initialises $\log Z_\theta$ at

$$\log Z_0 = \frac{1}{200} \sum_{i=1}^{200} \log R(x^{(i)}) + \log |\mathcal{X}|, \tag{17}$$

the mean log reward of its 200 calibration draws plus the logarithm of the number $|\mathcal{X}|$ of legal tuples. Our implementation of GFlowQEC-online replaces $\log R(x)$ by $\log \max(R(x), 10^{-9})$ in Eqs. (14) and (17). This floor binds on a scored tuple whose clamped score is below $10^{-9/\beta}$, which is 0.723 at $\beta = 64$.

The choice of β. A fixed rule sets the exponent from a sample of M uniform tuples $x^{(1)}, \ldots, x^{(M)}$ scored by the oracle. With $\mathcal{D}$ the best tenth of the sample by clamped score, β is the smallest listed value at which $\mathcal{D}$ holds four fifths of the total reward,

$$\beta = \min\Big\{ \beta' \in \{1, 2, 4, 8, 16, 27, 40, 64\} : \frac{\sum_{i \in \mathcal{D}} \max(s(x^{(i)}), 0.01)^{\beta'}}{\sum_{i=1}^{M} \max(s(x^{(i)}), 0.01)^{\beta'}} \ge 0.8 \Big\}, \tag{18}$$

or 64 if none qualifies. We set β on the reference sample rather than on the stored tuples, because the stored tuples are search outputs and would bias the rule.

Augmentation and the conditional policy. With σ a shuffle of the free slots of each sum and g, h drawn uniformly from G, the augmented tuple is

$$\tilde{x}_t = \begin{cases} g\, x_{\sigma(t)}\, g^{-1} & \text{for } t \text{ a slot of the } g_i, \\ h\, x_{\sigma(t)}\, h^{-1} & \text{for } t \text{ a slot of the } h_i. \end{cases} \tag{19}$$

In Eq. (19), σ reorders the two free slots of each sum independently, which gives 16 reorderings in the lifted-product family. The augmentation never translates a sum. Each $\tilde{x}$ only relabels the rows and columns of A and B, so it keeps the distances, d_A, d_B and bound score of x. The condition vector is

$$z = (z_+, z_-) = \Big(\frac{\max(\gamma, 0)}{1.5}, \frac{\max(-\gamma, 0)}{1.5}\Big), \tag{20}$$

which holds one non-negative number per sign of the bias, $(0.67, 0)$ at $\gamma = +1$ and $(0, 0.67)$ at $\gamma = -1$. The divisor 1.5 in Eq. (20) keeps both entries of z below one at $\gamma = \pm 1$. The conditional policy gives each bias its own normaliser,

$$\log Z_\theta(z) = \log Z_0 + f_\theta(z), \tag{21}$$

the sum of a fixed start $\log Z_0$, which is the same for both biases (Table B.1), and a learned correction $f_\theta(z)$ that starts at zero. With probability $1/2$, a training tuple $\tilde{x}$ is replaced by its mirror

$$M(x) = (h_1^{-1}, \ldots, h_5^{-1}, g_1^{-1}, \ldots, g_5^{-1}), \tag{22}$$

which builds a code equivalent to the mirror image of operation (f) and keeps both $g_1 = h_1 = 1$ and the outcome of the generation check. The mirror exchanges d_X with d_Z and, by item (f) of Appendix A.3, d_A with d_B. A mirrored tuple therefore scores at $-\gamma$ what the original scores at γ, $s(M(x), -\gamma) = s(x, \gamma)$, for the exact and the bound score alike. Since β is the same at γ and $-\gamma$, the policy trains on $(M(\tilde{x}), -\gamma)$ with the reward of x at γ, without an oracle call.

Table B.1. Hyperparameters of the networks behind Sec. 7 and Appendix E.2, as run. The sampler used at $n = 36$ and 72 is described in Appendix D.2. Adam runs at the PyTorch defaults ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$) with no weight decay, learning-rate schedule or gradient clipping.

Setting	GFlowQEC-online policy ($n = 180$, 300)	GFlowQEC-offline policy ($n = 180$)	Supervised screen ($n = 180$)	Soft Q-learning ($n = 180$, 300) and Munchausen DQN ($n = 180$)	PPO, actor and critic ($n = 180$, 300)
Network: architecture	multilayer perceptron	decoder-only transformer, causal mask	the policy's transformer without the causal mask	multilayer perceptron; target copy refreshed every 100 updates	two multilayer perceptrons
Input encoding	per-sum 0/1 vectors over G, slot one-hot; 180: 154; 300: 250	start vector + linear map of z, then filled-slot embeddings, each + a position vector	start vector + linear maps of z and of the relational counts (504), then all ten slot embeddings, each + a position vector	as online; 180: 154; 300: 250	as online; 180: 154; 300: 250
Hidden layers and widths	2×64	3 blocks, width 128, 4 heads, feed-forward 512, context 10	3 blocks, width 128, 4 heads, feed-forward 512, context 11	2×64	2×64 each
Activation	ReLU, $\max(0, \cdot)$	ReLU (feed-forward)	ReLU (feed-forward)	ReLU	ReLU
Normalisation	none	layer normalisation before each sub-layer; dropout 0	layer normalisation before each sub-layer; dropout 0	none	none
Output	$\lvert G\rvert$ logits per slot, Eq. (16), default initialisation; scalar $\log Z_\theta$	36 logits per slot; $\log Z_\theta(z)$, Eq. (21), f_θ linear, zero-initialised	one score on the start position, standardised per bias	$\lvert G\rvert$ soft Q-values per slot, zero-initialised	actor $\lvert G\rvert$ logits, critic one value; both zero-initialised
Parameters (measured)	180: 16,420; 300: 24,124; +1 for $\log Z_\theta$	605,863, incl. 3 of f_θ	666,113	180: 16,420; 300: 24,124; target copy not trained	actor as Q-network; critic 180: 14,145; 300: 20,289
Optimisation: optimiser	Adam [25]	Adam	Adam	Adam	Adam, actor and critic jointly
Learning rate	10^{-3}; $\log Z_\theta$: 10^{-1}	10^{-3}; f_θ: 10^{-2}	10^{-3}	10^{-3}	3×10^{-4}
Batch size	8 tuples	256, with replacement	256, with replacement	8 tuples	8 trajectories
Updates	3.125 per tuple in all: 1 per 8 new tuples, the rest replay[a]; 300, depth 127: 148,988–157,481	315,280; about 690 passes	252,224, chosen on held-out stored forms	as online[a]; 300: 132,576–147,801	4 epochs per batch; 300: 3,082–5,830 batches (entropy 0.01); 180: 401–435
Training data	200 calibration draws, then every tuple produced	116,972 stored forms	116,972 stored forms	as online	on-policy batches only
Augmentation	Eq. (19), replay only	Eq. (19), then Eq. (22) with probability 1/2	Eq. (19), then Eq. (22) with probability 1/2	as online	none
Replay buffer	all tuples, never emptied	–	–	as online	none
Loss or objective	Eq. (14), batch mean	Eq. (14) with $Z_\theta(z)$, batch mean	squared error, batch mean	soft Bellman squared error; Munchausen $\alpha = 0.15$, log-policy clip -85	clipped surrogate (clip 0.2, GAE $\lambda = 0.95$, discount 1) $+ 0.5$ value loss $-$ entropy
Sampling: exploration while training	per slot, uniform over the legal set with probability 0.05, else P_F	none, never sampled	–	as online, from softmax(Q/τ): $\tau = 1$ (soft Q), 1/0.85 (Munchausen)	own policy; entropy weight 0.01 or 0.1
Sampling at deployment	–	P_F, no mixture; first 2,000 forms (unranked, Appendix B.1) or top N of μN by $\log p_\theta$ (ranked, Sec. 5.4)	top N of $10N$ or $100N$ uniform candidates	–	–
Reward: β, exponent	Eq. (18) on the 200 draws; 180: 8, as offline; 300: 64, none qualifies	8 at $\gamma = \pm 1$: Eq. (18) on the reference sample and mirrors	–	300: 64, as online; 180: 8, as offline	as Q-network
$\log Z_\theta$ initialisation	Eq. (17)	Eq. (17) on the stored forms, mean over $\gamma = \pm 1$: 43.118; fixed	–	–	–
Run control: early stop	$> 2{,}000$ stale tuples in a row[b]	none	none	none; in the loop, as online	$> 2{,}000$ draws in a row without a paid call
Seeds and models trained	180: 0–9 per bias; 300: 0–5, 7–10; one model per run[c]	0–9, ten models	0–9, ten models	180: 0–9 per bias and learner; 300: 0–5, 7–10	180: 0–9 per bias; 300: 0–5, 7–10; per entropy weight

[a] Fresh tuples train first, in production order and unaugmented. Each replay update draws 8 buffer tuples uniformly without replacement. In the search loop the fractional remainder carries over, and the last iteration, cut short by the budget, takes no replay. [b] A stale tuple costs no paid call: it is answered from memory or fails the generation check. The count is checked after each iteration. [c] A seed fixes the initial parameters, the calibration draws, every draw of the policy and the seeding of the oracle's noise.

B.1 Offline training and deployment

Each stored form is kept as a single tuple with $g_1 = h_1 = 1$. For 45 of the 6,770 certified stored codes a ladder stopped early, and their stored distance is a lower bound. For the other 110,202 stored forms the stored distances are d_A and d_B, which makes the stored score an upper bound whenever $d_X \le d_B$ and $d_Z \le d_A$. Apart from setting β, the reference sample enters training only through the certification thresholds (10.0 at $\gamma = \pm 1$). At $\gamma = +1$, 3,359 stored forms score 10.0, and 756 of them reach it by the bound score alone (3,090 and 763 at $\gamma = -1$). Almost all of the forms scored by the bound alone were stored by runs at the opposite bias, which certify only at their own bias. So 22.5 and 24.7 per cent of the forms trained at the top reward, at the two biases, have an unknown exact score. None of the canonical forms of the 666 codes of the reference sample, or of their mirror images, is among the stored forms, although a stored code can still be equivalent to one of those codes.

In a pilot at $n = 180$ in which both networks trained for 1,699 steps (the deployed policy trains for 315,280), the transformer did about 1.2 to 1.3 times as well as a perceptron with the hidden layers used online, which is within noise. The pilot cannot tell whether the architecture or the longer training improves the policy. We also trained a variant of the policy that additionally reads the relational counts of Appendix B.2. We do not use it, because in the ranked deployment at $\mu = 10$ it found 33, 0 and 0 novel certified top-scoring codes in the three seeds at $\gamma = +1$, against 308, 297 and 333 for the deployed policy (3, 0 and 0 against 294, 321 and 311 at $\gamma = -1$).

An earlier, unranked deployment paid one oracle call for each of the policy's first 2,000 distinct canonical forms, in the order drawn.

At zero loss, $Z_\theta(z)\, p_\theta(x \mid z)$ would equal the stored tuple's reward, so

$$\log p_\theta(x \mid z) = \beta \log \max(s, 0.01) - \log Z_\theta(z), \tag{23}$$

where s is the stored score. At a fixed bias, the log-probability would then rank the stored tuples in the order of their stored scores, which is what lets the ranked deployment use $\log p_\theta(x \mid z)$ as a stand-in for the score (Sec. 5.4). Draws are not checked against the stored forms, so a form that is sent may already be stored, and it is still paid for.

B.2 Relational counts

The supervised screen (Appendix C.1) also reads relational counts, which do not change when a sum is translated. For each sum, they count how often each element of G occurs as the product gg'^{-1} of two distinct elements of the sum, the identity included ($g^{-1}g'$ for the h_i). For each side (the g_i or the h_i), they count these products in each conjugacy class, the set of elements that the conjugation of operation (d) reaches from one element. Finally, for each pair of classes, they count the pairs of products, one from each side, that fall in that pair.

C Protocol details and statistical procedures

Budgets. The exceptions to the budget of Appendix E.1 are ten runs of Bayesian optimisation at $n = 300$, which have a budget of 200 calls. They make too few proposals for W, so they are graded by their best code.

Measures. The neighbourhood count groups certified top-scoring codes, novel or not, by the half of the tuple, $(g_1, \ldots, g_5)$ or $(h_1, \ldots, h_5)$, that limits their distance against the likelier flip. At $\gamma = +1$ a top-scoring code needs $d_X \geq 10$, and d_X is capped by d_B, which depends only on the b half, h_1 to h_5. We therefore count two top-scoring codes as one neighbourhood when their b halves agree up to conjugating every element of one by a single group element, however different their a halves. At $\gamma = -1$ the a half plays this role. At $n = 300$ the reference rate y^* is set per seed, as the median of the re-estimated rates pooled over five earlier runs with that seed, one stream each: the GFlowQEC-online run at depth 31 and its two variants of Table C.1, local search alone at depth 31, and tuned PT. The ten values lie between 0.004228 and 0.004245.

C.1 Comparison methods as run

Table C.1. Comparison methods and controls as run, beyond Sec. 6.1. The networks are in Table B.1.

Method	n	Settings
Uniform sampling	all	redraws until the generation check passes
Parallel tempering	all	eight chains at geometrically spaced temperatures, from paid uniform draws or stored forms (which never count); a one-slot uniform move; a drop $\Delta > 0$ accepted with probability $e^{-\Delta/\mathcal{T}}$; every ten sweeps neighbours exchange with probability $\min(1, \exp[(1/\mathcal{T} - 1/\mathcal{T}')(s' - s)])$
Tuned PT range	all	180: 0.25–2.0 (better of two, all 26 bias and seed pairs); 300: 0.01–0.15 (best of five on packed yield, the coldest, the only one run on W)
MAP-Elites	all	200 uniform draws, then PT's move on a code of a random occupied bin; the child takes an empty bin or beats its code; 180: five bins of $d_A - d_B$ (edges -3, -1, 1, 3), or these $\times$ four of k (36, 38, 40, other); 300: four of k (60, 62, 66, other) $\times$ eight octiles of bit-flip minus phase-flip failure fraction
Soft Q-learning, Munchausen DQN	180, 300	after 200 paid uniform draws, a soft Q-value per allowed element on the reward $\log R(x)$; Munchausen DQN with the constants of Tiapkin et al. [47].
Soft Q-learning, trained offline	180	the network, stored evaluations, training steps, batch and deployment rule of GFlowQEC-offline; Eq. (14) replaced by the soft Bellman squared error (each slot's Q-value regressed on the next slot's log-sum-exp over the legal set, the last on $\log R(x)$), against a target copy refreshed every 100 updates
PPO	180, 300	reward $\beta \log \max(s, 0.01)$ less its mean over 200 initial draws; generalised advantage estimation [41]; entropy weights 0.01, our implementation's default, and 0.1; stops after 2,000 draws without a new canonical form; no local move
BO	180, 300	one-hot over the ten group elements of the canonical form's tuple; Matérn 5/2 Gaussian process; expected improvement [21], margin 0.01, over 2,000 uniform candidates; 200 calls: 20 initial (Chengyu et al.'s public code: 220 [10]); 40,000 calls: 200 initial, refit every 50; at $n = 180$, 1,000 calls: 200 initial
Supervised screen	180	the policy's network without the causal mask, one score on the start position and the relational counts of Appendix B.2 added to its start vector (666,113 parameters); squared error on the standardised stored score at $\gamma = \pm 1$, the policy's augmentation, batches and seeds; 252,224 updates, chosen among four variants and twenty checkpoints by held-out precision on a tenth of the stored forms, then trained on all of them; pays for its top 1,000 of 10 or 100 times as many uniform candidates
Pilot screen	180	multilayer perceptron, two hidden layers of 64 (45,889 parameters), batch 64, 315,280 updates; trained at $\gamma = \pm \log_{10} 30$, $\pm \log_{10} 3$ and 0; reads the relational counts of Appendix B.2
Supervised screen	300	trained on the run's own paid calls: 200 uniform draws first, then in each of 1,990 rounds the network (the relational counts of Appendix B.2, two hidden layers of 64, 96,449 parameters, per-side conjugation as in Eq. (19), then the mirror of Eq. (22) with probability 1/2) is refit by 160 warm-started Adam steps on the paid scores and pays for its top 20 of 2,000 fresh uniform candidates
Objective swap	300	Eq. (14) replaced by the mean over free slots of $(\log Z_\theta + \ell_\theta(x_t; x_{<t}) - \log R(x))^2$, regressing each chosen logit (Eq. (16)) on the log reward
Plain sampler	300	the policy trained online without local search
Variants behind y^*	300	$L = 31$; uniform local steps, or each proposal scored on its current tuple's errors and, if accepted, paid again on its own

Additional runs. We also made the following runs.

(i) Pilot and development runs that chose settings: at $n = 300$, GFlowQEC-online at depths 0, 1, 3 and 7, earlier samplers, further depth-31 variants and a hybrid that runs PT and then the policy.
(ii) At $n = 300$, four runs of 10,000 calls each that decided whether to use common random numbers.
(iii) At $n = 180$, deployments of an untrained policy and of policies with shorter training.

C.2 Statistical procedures

Packed yield and near-duplicates. Packed yield was the measure we used at $n = 300$ before W. It counted the paid codes whose in-run ler_{pq} was at most 0.003567, the first percentile of 26,281 earlier uniform draws, and kept a code only if its canonical form's tuple differed from every code already kept in more than two group elements. The same rule gives the count of good proposals with near-duplicates counted once, which is W times the share of proposals that the rule keeps in a greedy pass in call order.

Table C.2. Statistical procedures. Every interval is at 95 per cent, and no correction for multiple comparisons is applied.

Procedure	Definition	Where used
Seed count	paired seeds in which one method leads; under a fair coin per seed, ten of ten has probability 0.001 and at least eight of ten 0.055	everywhere
Paired t interval and test	mean per-seed difference in per cent of the comparison method's mean; assumes roughly normal differences; p-values only in Appendix E.2	W at $n = 300$
Log-ratio interval, Eq. (24)	ratio of pooled counts	$n = 180$

To compare a count κ_1 out of ν_1 paid calls with a count κ_2 out of ν_2, both counts nonzero, we use the log-ratio interval of Katz et al. [22],

$$\frac{\kappa_1/\nu_1}{\kappa_2/\nu_2}\, e^{\pm 1.96\,\omega}, \qquad \omega = \sqrt{\frac{1}{\kappa_1} - \frac{1}{\nu_1} + \frac{1}{\kappa_2} - \frac{1}{\nu_2}}, \tag{24}$$

where ω is the standard error of the log ratio. The interval treats every paid call as independent and so leaves out the variation between seeds (separately trained models at $n = 180$) and within a chain. The per-seed counts are therefore printed beside it.

D Further results

This appendix gives the per-seed results behind Sec. 7, together with the other runs.

D.1 GFlowQEC-offline at $n = 180$

Table D.1 gives the per-seed counts behind Sec. 7. On the inequivalent count, GFlowQEC-offline found 2.40 $[2.23, 2.58]$ and 2.80 $[2.59, 3.02]$ times as many top-scoring codes as tuned PT, pooled over the ten seeds, and between 1.72 and 4.85 and between 2.11 and 6.94 times as many in the single seeds. Compared with the supervised screen, the policy ran the exact ladders on slightly more codes for each novel top-scoring code it found (1.41 against 1.40 at $\gamma = +1$ and 1.43 against 1.40 at $\gamma = -1$). The unranked deployment (Appendix B.1) paid for the policy's draws in the order they were drawn. Over all its 2,000 calls it trailed, per paid call and in every seed at either bias, the earlier and smaller pilot screen, a multilayer perceptron trained on the same stored scores at other biases (Table D.1, note b).

Table D.1. Certified top-scoring codes at $n = 180$ in the first 1,000 paid calls of each run, novel ones only in the rows above plain tuned PT, the seeds in the Seeds column (0 to 9, except the pilot screen at 100 times: 1 to 9). Ratio: the ranked deployment's pooled count at $\mu = 10$ over the row's, interval by Eq. (24). Nbhd: the same for neighbourhoods. Ratio carries its interval when below 10. Ratio and Nbhd are blank for the screens at 100 times, whose oversample the deployment at $\mu = 10$ does not match. Above the rule, a code counts only if its canonical form is not among the stored forms. Below it, every certified top-scoring code counts (Sec. 6.2).

		$\gamma = +1$			$\gamma = -1$		
Method	Seeds	Codes	Ratio	Nbhd	Codes	Ratio	Nbhd
GFlowQEC-offline, ranked, $\mu = 10$	10	308, 297, 333, 283, 302, 314, 321, 317, 337, 321			294, 321, 311, 298, 342, 307, 296, 321, 338, 346		
Supervised screen, 10 times	10	99, 69, 89, 81, 101, 101, 92, 85, 71, 78	3.62 [3.37, 3.88]	2.58	84, 85, 94, 77, 93, 80, 86, 87, 86, 68	3.78 [3.52, 4.06]	2.63
Policy as a ranker[a]	10	69, 57, 62, 64, 73, 75, 69, 66, 54, 63	0.75 [0.68, 0.83]	0.79	61, 66, 78, 59, 75, 74, 65, 73, 72, 59	0.81 [0.74, 0.89]	0.84
Ranked, $\mu = 100$[a]	10	273, 338, 400, 185, 235, 258, 301, 276, 261, 236	0.45 [0.44, 0.47]	0.51	336, 463, 488, 229, 290, 360, 367, 421, 434, 349	0.62 [0.61, 0.64]	0.60
Supervised screen, 100 times	10	582, 642, 636, 615, 660, 626, 645, 643, 501, 563			500, 644, 590, 611, 659, 621, 625, 626, 558, 555		
Pilot screen, 10 times	10	71, 61, 66, 66, 80, 73, 75, 55, 60, 56	4.73 [4.37, 5.11]	3.44	82, 64, 74, 63, 61, 62, 56, 55, 66, 60	4.94 [4.56, 5.35]	3.46
Pilot screen, 100 times	9	293, 266, 280, 293, 243, 294, 272, 238, 274			312, 294, 240, 262, 235, 298, 305, 258, 288		
Unranked deployment[b]	10	49, 61, 60, 42, 73, 44, 62, 47, 54, 56	5.72 [5.24, 6.23]	3.81	50, 49, 53, 58, 52, 53, 52, 46, 46, 57	6.15 [5.63, 6.72]	4.00
Soft Q-learning, trained offline	10	256, 325, 313, 303, 325, 316, 353, 327, 370, 286	0.99 [0.95, 1.03]	1.03	338, 314, 342, 380, 399, 389, 404, 364, 390, 324	0.87 [0.84, 0.91]	0.97
Tuned PT started from the stored forms	10	106, 147, 113, 124, 115, 113, 103, 114, 90, 165	2.63 [2.48, 2.80]	6.92	122, 131, 96, 129, 116, 109, 143, 122, 108, 158	2.57 [2.42, 2.73]	6.97
Tuned PT	10	111, 108, 96, 75, 103, 136, 95, 52, 91, 105	3.22 [3.02, 3.44]	8.41	74, 109, 102, 101, 63, 72, 109, 74, 34, 97	3.80 [3.54, 4.08]	11
MAP-Elites, 5 bins	10	91, 92, 79, 150, 108, 124, 57, 115, 110, 108	3.03 [2.84, 3.23]	13	78, 95, 87, 96, 97, 92, 91, 97, 87, 102	3.44 [3.22, 3.68]	17
MAP-Elites, 20 bins	10	51, 60, 43, 59, 46, 41, 32, 36, 32, 49	6.98 [6.35, 7.67]	15	36, 40, 37, 50, 50, 42, 38, 43, 29, 40	7.84 [7.09, 8.66]	17
PPO, entropy weight 0.1	10	10, 11, 8, 17, 5, 14, 12, 7, 12, 7	30	19	12, 12, 11, 12, 11, 7, 8, 8, 5, 4	35	22
PPO, entropy weight 0.01	10	8, 11, 5, 14, 11, 17, 11, 6, 7, 7	32	20	6, 8, 12, 9, 7, 7, 8, 8, 7, 11	38	23
Soft Q-learning, online	10	16, 9, 9, 7, 12, 12, 6, 14, 11, 11	29	18	10, 8, 8, 15, 11, 7, 11, 5, 5, 10	35	22
Munchausen DQN	10	12, 10, 12, 10, 10, 9, 4, 14, 9, 12	31	19	12, 12, 9, 18, 13, 7, 12, 13, 8, 12	27	17
GFlowQEC-online, Munchausen DQN as learner	10	16, 20, 25, 19, 18, 11, 11, 16, 21, 12	19	17	27, 14, 12, 14, 12, 13, 9, 20, 13, 10	22	21
GFlowQEC-online, depth 7	10	17, 14, 19, 18, 13, 12, 6, 14, 17, 25	20	18	19, 12, 15, 14, 16, 14, 8, 13, 29, 10	21	20
Uniform sampling	10	10, 9, 14, 16, 13, 9, 6, 9, 15, 14	27	17	10, 11, 7, 6, 6, 7, 12, 8, 9, 8	38	23
BO, one-hot encoding	10	38, 23, 41, 36, 40, 86, 60, 89, 36, 47	6.32 [5.77, 6.92]	11	50, 16, 82, 64, 10, 53, 103, 76, 36, 37	6.02 [5.52, 6.58]	15

[a] Ratio of this row over the screen at the same μ (10 for the ranker). [b] Over all its 2,000 paid calls the unranked deployment's rate was 0.80 [0.73, 0.88] and 0.81 [0.73, 0.89] times the pilot screen's (Eq. (24)), below it in every seed.

Sorting the stored evaluations by k, the number of logical qubits, would raise their top-scoring share by at most 1.4 times, so k does not single out the top-scoring codes.

D.2 Proportional sampling at 36 and 72 physical qubits

Proportional sampling can be tested only where the exact target is known, which is at $n = 36$ and 72. We tested it on an earlier, simpler sampler, a multilayer perceptron that chooses each sum's elements in any order and is trained by trajectory balance with $\beta = 1$ on the whole family. We measure the total variation distance between the sampler's draws and the exact target of Eq. (11), with both binned by the family's score values. This distance is half the sum, over the values, of the absolute difference between the two shares, so it runs from 0 for identical distributions to 1 for distributions with no value in common.

At $n = 36$, where the family has eight score values, the sampler's 10,000 draws lie at a distance of 0.089 from the target, against 0.72 for a uniform sampler. Three retrainings gave 0.092 to 0.100 and put about a third too little mass on the highest score value, which means that this small sampler under-samples the best codes. At $n = 72$, after 360,000 training steps, the sampler lies at 0.115 to 0.124 over three seeds, against 0.76 for a uniform sampler, and its learned $\log Z$ is within 0.131 of the exact value. It gives the highest score value about half of its target mass (0.47 to 0.54), and training ten times longer or replaying high-scoring codes more often closed only part of that gap. A larger or better-suited network might close it, but we did not test one.

We did not pursue this further, because GFlowQEC-offline pays for the most probable of many draws rather than for draws in proportion. That ranking found more than five and a half times as many top-scoring codes as paying for the draws in the order drawn (Table D.1). The policy of Sec. 5 has not been tested in this way, and a test is planned at $n = 72$.

E GFlowQEC-online at $n = 300$

In the online setting the policy of Sec. 5 is trained while it searches, as the proposal distribution of a local search in the manner of local-search GFlowNets [24]. A run starts with 200 calibration draws, uniform tuples scored by the oracle, which set β by Eq. (18) with $M = 200$ and fix the floor reward $R_{\min}$ as the smallest reward among them. At $n = 300$ the rule gave $\beta = 64$, the value it takes when no candidate exponent qualifies. Each iteration then draws four fresh tuples from the policy, and each one that passes the generation check is scored and starts a chain of L local steps, where L is the depth of the search. A local step re-samples one free slot, picked uniformly at random, from the policy, and the chain moves to the proposal only when its score is strictly higher, the deterministic filter of Kim et al. [24]. Every tuple, accepted or rejected, enters the replay buffer with its reward ($R_{\min}$ for a tuple that fails the check) and trains the policy by trajectory balance (Sec. 5.3), and every slot the policy samples

is drawn uniformly from the legal set with probability 0.05. Because the chain only climbs, its tuples are not drawn in proportion to R, although the training target is unchanged. Here the network is a multilayer perceptron (Table B.1). The baseline we call local search alone runs the same chains without a policy, so every fresh tuple and every re-sampled slot is drawn uniformly.

E.1 Score, budget and grading

We chose the lifted-product family over $Z_{10} \times S_3$ because the score varies widely from code to code, no algebraic quantity that is cheap to compute predicts it, uniform sampling yields far fewer good codes per paid call than a search does, and the space, at most about 3×10^8 codes up to equivalence, is too large to enumerate. The score comes from a logical error rate under code-capacity depolarizing noise, in which each data qubit independently suffers X, Y or Z with probability $p/3$ each, with $p = 0.07$, and every check is measured perfectly. BP-OSD [35,40,39] decodes the bit-flip part of the error from the Z checks and the phase-flip part from the X checks. The block failure rate ler is the fraction of failed shots among S shots (S is unrelated to S_3), and with the per-logical-qubit rate $\mathrm{ler}_{pq} = 1 - (1 - \mathrm{ler})^{1/k}$ the score is

$$s(x) = -\ln\Big(\mathrm{ler}_{pq} + \frac{1}{kS}\Big), \tag{25}$$

where the added term keeps the score finite when no shot fails. No code at this size was certified. The sampled errors are seeded in one of two ways. With one stream per code they depend on the run's seed and the code's canonical form. Under common random numbers [17] every call of a run draws the same errors, which compares a proposal with its current tuple under less noise. A run has 40,000 paid calls, initial uniform draws included (exceptions in Appendix C). We grade a run by its good proposals $W = VF$. Here V is the number of its proposals, the paid calls after its initial draws (for GFlowQEC-online, its local steps), and F is the fraction of 800 of them, re-estimated on fresh errors, whose ler_{pq} is at most a reference rate y^* of about 0.00424, set per seed from five earlier reference runs (Appendix C). Two proposals are near-duplicates when their canonical forms differ in at most two group elements (Appendix C.2).

E.2 Results at $n = 300$

At depth $L = 127$, GFlowQEC-online made more good proposals per paid call than tuned PT in all ten seeds (Table E.1), 10.0 per cent more [6.7, 13.3] with one stream per code (two-sided $p = 0.00008$, paired t test) and 12.8 per cent [9.7, 16.0] under common random numbers. It led local search alone, the same chains without the policy, by 8.9 per cent [5.5, 12.2] in ten of ten seeds ($p = 0.0002$), and this lead is what the policy adds to the local move. It also led the supervised screen trained on the run's own paid calls by 10.0 per cent [5.7, 14.3] in ten of ten seeds ($p = 0.0005$). MAP-Elites and PPO trailed it in all ten seeds,

as did soft Q-learning (W = 10,686) and PPO at entropy weight 0.01 (9,889), whose runs all stopped early after 2,000 draws in a row without a new code to pay for. The objective swap, which regresses each chosen logit on the log reward (Appendix C.1), was not separated from GFlowQEC-online (+0.73 per cent $[-2.0, +3.5]$ at $L = 127$), so we do not attribute the lead to trajectory balance. Nor does k explain it. Codes with $k = 60$ make up 70 to 73 per cent of the proposals of GFlowQEC-online, tuned PT and uniform sampling alike, and within that class GFlowQEC-online's re-estimates reached the reference rate in 62.1 per cent of cases, against 55.3 for tuned PT and 26.1 for uniform sampling.

Table E.1. Comparisons at 300 physical qubits. Good proposals per run, W, as the mean of ten seeds (0 to 5 and 7 to 10, seed 6 was used for tuning) of 40,000 paid calls. Near-duplicates are proposals whose canonical forms differ in at most two group elements (Appendix E.1). The two groups seed the sampled errors differently (Appendix E.1). Bold: best per column within a group.

Method	Good proposals per run	Good proposals with near-duplicates counted once
One stream per code		
GFlowQEC-online	**23,255**	3,478
Local search alone	21,362	3,270
Tuned PT	21,146	6,076
MAP-Elites	16,258	7,644
PPO, entropy weight 0.1	11,403	11,341
Uniform sampling	9,360	9,339
Supervised screen, trained on the run's own paid calls	21,139	**21,014**
Common random numbers		
GFlowQEC-online	**24,633**	4,016
Tuned PT	21,831	7,162
Bayesian optimisation, one-hot encoding	18,492	**13,274**

GFlowQEC-online and local search alone run at depth 127. Bayesian optimisation ran under common random numbers only.

With near-duplicates counted once, the lead reverses. Tuned PT, MAP-Elites, uniform sampling, PPO at entropy weight 0.1 and the supervised screen each led GFlowQEC-online in all ten seeds, and under common random numbers Bayesian optimisation led it too. Its chains of 127 one-element changes stay among close relatives of their first tuple, so dropping near-duplicates keeps about 0.15 of its proposals, against 0.72 to 1.0 for a method that draws every tuple afresh. Local search alone keeps the same share, and GFlowQEC-online led it with near-duplicates counted once in eight of ten seeds, so the deficit belongs to the chain rather than to the policy.

The best codes do not separate the methods. For six configurations (GFlowQEC-online and tuned PT under both seeding modes, local search alone at $L = 127$, and the screen), the lowest re-estimated rates among the 200 highest-scoring codes of each run lie between 0.00335 and 0.00350 per logical qubit, and their 95 per cent Wilson intervals [51] overlap. In every seed except seed 1, the best codes of Bayesian optimisation at 200 calls, of uniform sampling and tuned PT at their first 200 calls, and of GFlowQEC-online at 40,000 calls under common random numbers lie within one Wilson interval of each other.

Choices made after reading runs. At $n = 300$ we changed the measure from packed yield (Appendix C.2) to good proposals after reading the depth-31 and tuned PT runs, and this change went against GFlowQEC-online. We set the depth to 127 after its runs at depth 31 had trailed tuned PT on good proposals in six of six seeds, and the temperature range of tuned PT, chosen on packed yield, was the best of five on the same seeds. The p values do not account for these choices.